

Arbeitsexposé

KI-gestützte 3D-Rekonstruktion linearer Infrastruktur: Evaluierung einer Docker-basierten Scan-to-BIM Pipeline mittels SAM 3 und Gaussian Splatting

Bachelor- und Projektarbeit

9. Juni 2026

1 Einleitung und Forschungsfrage

Die präzise Erfassung schwer zu modellierender, extrem dünner Objekte (wie Stromerkabel in Baugraben) stellt für klassische photogrammetrische Verfahren (SfM) eine große Herausforderung dar. Ziel dieser Arbeit ist die Entwicklung und Evaluierung eines "Scan-to-BIM"-Workflows, der die Vorzüge der 2D-KI-Segmentierung (Segment Anything Model 3) mit modernsten 3D-Rekonstruktionsmethoden (3D Gaussian Splatting via SuGaR) verbindet.

Wie aktuelle Forschungen belegen, erzeugen 3D Gaussian Splatting Algorithmen durch mathematische Wahrscheinlichkeitsfunktionen hochgradig detailreiche, fotorealistische Visualisierungen bei geringer Rechenlast. Sie werden zunehmend im Ingenieurwesen und zur Generierung von Orthofotos (z.B. Integration in ArcGIS Pro) eingesetzt. Jedoch fehlt 3DGS-Modellen die inhärente Georeferenzierung. Die zentrale Forschungsfrage lautet daher: *Lässt sich die 3D-Rekonstruktion linearer Infrastruktur durch eine vorgelagerte 2D-Segmentierung und ein objektspezifisches 3DGS-Training (Segment-then-Splat) so optimieren und georeferenzieren, dass strenge ingenieurstechnische Toleranzen eingehalten werden?*

2 Praxisbezug und Erfolgskriterien

Die Pipeline wird direkt für den Anwendungsfall der TenneT (Übertragungsnetzbetreiber) konzipiert.

- **Geometrische Toleranz:** Eine Abweichung von maximal ± 10 cm in Lage und Höhe gegenüber der realen Kabeltrasse.
- **Georeferenzierung & GIS-Integration:** Das finale 3D-Modell muss präzise in einem UTM-Koordinatensystem vorliegen, um den direkten Import in Geoinformationssysteme (z.B. ArcGIS Pro) zu gewährleisten.
- **Wirtschaftliche Evaluierung:** Eine Kostengegenüberstellung des Drohnen- und Serverrechenaufwands im Vergleich zur klassischen, terrestrischen Vermessung.

3 Methodik und Architektur (Workflow v3)

Der Ansatz basiert auf einer klaren Trennung von 2D-Bildverarbeitung und 3D-Meshing.

3.1 Software-Repositories

- **SAM 3 (Meta)**: Zuständig für Concept-Prompting und Video-Tracking.
- **COLMAP**: Generierung der Kameraposen auf Basis der *unmaskierten* Originalbilder. Die Pipeline nutzt ausschließlich das schnelle *Structure from Motion* (SfM) von COLMAP. Die rechenintensive dichte Rekonstruktion (MVS) wird vollständig übersprungen.
- **Segment-then-Splat (STS)**: Übernimmt das objektspezifische 3DGS-Training (Lu et al., NeurIPS 2025). STS teilt die initialen COLMAP-Gaussians anhand der SAM 3 Masken in objektspezifische Sets auf und trainiert diese mit einem dedizierten Object-specific Loss ($\mathcal{L}_{obj}$), ohne die Ground-Truth-Bilder zu manipulieren. Das Ergebnis ist ein vollständiges 3DGS-Modell, in dem jeder Gaussian eine Object-ID trägt. SAM 3 ersetzt hierbei das intern vorgesehene AutoSeg-SAM2.
- **SuGaR**: Surface extraction with Gaussian Splatting. Übernimmt den vortrainierten STS-Checkpoint und wendet geometrische Regularisierung (DN-Consistency, Normal Alignment) sowie Poisson Surface Reconstruction an. SuGaR zwingt die von STS bereits sauber zugewiesenen Gaussians dazu, sich flach an die reale 3D-Oberfläche anzuschmiegen, und extrahiert daraus ein hochqualitatives Mesh.
- **DGtal**: Open-Source Geometrie-Bibliothek zur finalen Centerline-Extraktion. Sie wandelt die Oberfläche des Kabel-Sub-Meshes in eine räumliche 1D-Kurve (Centerline bzw. Scheitelachse mit lokalen Koordinaten) um.

3.2 Pipeline-Struktur: Temporale Konsistenz vs. SAHI

Für die Erkennung extrem dünner Kabel in hochauflösenden 4K-Bildern bietet sich theoretisch *Slicing Aided Hyper Inference* (SAHI) an. SAHI zerschneidet das Bild in kleinere Teilbereiche (Patches), wodurch verhindert wird, dass dünne Kabel beim Downscaling auf die native Modellauflösung von SAM 3 "verschluckt" werden.

Dennoch wurde der primäre Fokus für diese Pipeline bewusst auf den **SAM 3 Video Predictor** gelegt. Der methodische Nachteil von SAHI ist die fehlende temporale Konsistenz: Da SAHI jedes Einzelbild isoliert betrachtet, "flackern" die erzeugten Masken von Frame zu Frame, wenn sich Lichteinfall oder Blickwinkel leicht ändern. 3D Gaussian Splatting erfordert jedoch zwingend formstabile, konsistente Masken über alle Kameraperspektiven hinweg, da sonst fehlerhafte Artefakte in den 3D-Raum projiziert werden. Der SAM 3 Video Predictor löst dieses Problem durch ein Memory-Modul, welches die Kabel-Pixel über die Zeit hinweg verfolgt und dadurch flackerfreie, temporär konsistente Masken liefert. SAHI dient in dieser Pipeline lediglich als Fallback-Route für spezifische Bildausschnitte, in denen das temporale Tracking abreißen sollte.

4 Docker-Infrastruktur und Orchestrierung

Ein Kernproblem moderner KI-Pipelines sind tiefgreifende Versionskonflikte zwischen den benötigten Nvidia-Grafikkartentreibern (CUDA). Dieses Problem wird durch eine **docker-compose** Architektur vollständig gelöst. Die Pipeline isoliert die Anwendungen in voneinander getrennte Container, während alle erzeugten Daten über **Docker Bind Mounts (Volumes)** zentral und persistent auf dem Host-System (Windows/Linux) gespeichert werden.

Um die Automatisierung zu maximieren, wird die Pipeline über ein zentrales Master-Skript (*One-Click-Orchestration*) gesteuert. Dieses Skript stößt die Container sequenziell an, muss jedoch für manuelle Arbeitsschritte pausieren. Es kommen fünf dedizierte Container zum Einsatz:

- **Container A (Vorverarbeitung):** Beinhaltet PyTorch 2.x und SAM 3. Benötigt das Nvidia Container Toolkit (CUDA ≥ 12.6).
- **Container B (SfM):** Ein offizielles NVIDIA-Image für COLMAP (CUDA 11.8 oder 12.1). *Nach Ausführung dieses Containers pausiert das Master-Skript automatisch (Breakpoint). Der Nutzer öffnet CloudCompare auf dem Host-System, pickt die GCPs und speichert die Transformationsmatrix. Anschließend wird das Skript mit "Enter" fortgesetzt.*
- **Container C (Object-specific 3DGS):** Beinhaltet Segment-then-Splat (STS). Empfängt die SAM 3 Masken und die COLMAP-Kameraposen und führt das objektspezifische 3DGS-Training mit dem nativen Object-specific Loss ($\mathcal{L}_{obj}$) durch. Benötigt CUDA 12.1 (PyTorch 2.3.1). SAM 3 ersetzt hierbei das intern vorgesehene AutoSeg-SAM2 als Masken-Frontend.
- **Zwischenschritt 3.5 (Kabel-Isolierung & Qualitätsfilterung):** Ein dediziertes Python-Skript `filter_cable_pc.py` filtert die dichte STS 3D-Punktwolke direkt vor der Übergabe an das Meshing-Tool. Es extrahiert gezielt die Kabel-Vordergrund-Splats auf Basis der gelernten Objekt-IDs (Standardwert auf mittlerer Ebene `obj_id_m == 1`) und wendet zusätzlich hochentwickelte Opazitäts- und Farbschwellenwert-Filter an, womit transparente Floater sowie schwarze Splat-Gase („Rauch-Artefakte“) mathematisch bereinigt werden, um SuGaR-Artefakte drastisch zu reduzieren.
- **Container D (Meshing):** Beinhaltet SuGaR. Lädt den gefilterten, isolierten Kabel-Checkpoint und wendet die geometrische Regularisierung (DN-Consistency) sowie Poisson Surface Reconstruction an. Wegen tiefer Abhängigkeiten zum `diff-gaussian-rasterization` Modul wird hier strikt CUDA 11.8 erzwungen. Das intern geforderte Conda-Environment wird systemweit über eine Anpassung der `ENV PATH` Variable im Dockerfile nativ zugänglich gemacht.
- **Container E (Post-Processing):** Ein reiner **CPU-Container** (ohne GPU/CUDA-Abhängigkeit). Beinhaltet die C++ Bibliothek DGtal sowie die GDAL-Tools (`ogr2ogr`). Durch die Auslagerung dieser CPU-basierten Nachbearbeitung in einen eigenen Microservice bleibt die Architektur extrem leichtgewichtig, da keine massiven Nvidia-Images geladen werden müssen.

5 Risikomanagement und Datenhandling

Mehrere kritische Punkte und prozessuale Hürden erfordern besondere Aufmerksamkeit:

1. **Speichermanagement:** Drohnenvideos in 4K bei 60 FPS erzeugen massive Datenmengen. Das Konzept sieht vor, dass diese temporär im nativen Linux-Dateisystem liegen und nach der 6 FPS Extraktion gelöscht werden, um I/O-Engpässe unter WSL2 zu vermeiden.

2. **SAM 3 Tracking-Stabilität (VRAM) und Modellkonsistenz:**

In der praktischen Ausführung zeigte sich ein nichtlinear wachsender VRAM-Bedarf im `propagate_in_video`-Pfad (insb. während `F.interpolate` in der Output-Erzeugung). Dadurch trat ein `CUDA out of memory` nicht deterministisch auf (z. B. um Frame 45, bei anderen Läufen früher oder später), obwohl dieselbe Eingabe verwendet wurde. Ursache ist die Kombination aus langem Sequenzgedächtnis (Memory-Modul), fragmentiertem Allocator-Zustand und voller HD-Auflösung über viele Frames.

Als technische Gegenmaßnahme wurde die Propagation auf feste Zeitfenster (Chunking) umgestellt: Jeder Lauf verarbeitet nur ein Segment mit `start_frame_index` und `max_frame_num_to_track`; die Masken werden unmittelbar pro Frame persistiert, statt alle Outputs bis zum Ende im RAM/VRAM vorzuhalten. Zusammen mit Offloading-Flags (Video/State auf CPU) stabilisiert dies die Pipeline auf 16-GB-Karten.

Ein zweiter, davon getrennter Befund ist die **Gewichts-/Architektur-Konsistenz**: Trotz stabilem Lauf können aufgrund von `Missing keys/Unexpected keys` beim Laden des SAM-Checkpoints formal gültige, aber inhaltlich leere Masken entstehen. Daher wird als Quality-Gate vor Übergabe an STS verbindlich geprüft: (i) Anteil nichtleerer Masken, (ii) temporale Kontinuität über benachbarte Frames, (iii) Sichtprüfung an Stichproben. Nur bei erfüllten Kriterien werden Masken in Container C übernommen.

3. **[GELÖST: SAM 3.1 Video-Tracking stabilisiert, Stand 06/2026]**

Die anfängliche Instabilität des SAM 3.1 Video-Pfades konnte durch drei gezielte Maßnahmen gelöst werden, was eine unterbrechungsfreie Voll-Propagation über alle 100 Frames auf einer 16-GB-GPU ermöglichte: (i) **VRAM-Limitierung durch sequenzielles Vision-Language-Matching**: Die Aktivierung von `batched_grounding_batch_size = 1` (anstelle von standardmäßigem Batching nach Text-Prompts) reduzierte den VRAM-Verbrauch von über 12 GB auf unter 5 GB, wodurch jegliche `CUDA out of memory`-Fehler verhindert wurden. Zusätzlich wurde die PyTorch-Compilation via `TORCH_COMPILE_DISABLE=1` deaktiviert, um massive RAM/VRAM-Spitzen vorzubeugen.

(ii) **Behebung des Shape-Assertions-Fehlers**: Der zuvor auftretende Fehler `AssertionError: pos_pred_mask.shape[0] == 1` bei der Chunk-Verarbeitung wurde durch Rückkehr zum globalen Video-Inferenz-Modus unter Verwendung der Standard-Konditionierungsparameter (`max_cond_frames_in_attn` unlimitiert) gelöst. (iii) **Datensatz-Kompatibilität bei der Masken-Persistierung**: SAM 3.1 Multiplex liefert den Vorhersage-Stream nicht in Form geschachtelter Dictionaries wie reguläre SAM-Modelle, sondern im Multiplex-spezifischen Format, d.h. als Dictionary mit den Hauptschlüsseln `out_obj_ids` und `out_binary_masks`. Ein angepasster Masken-Parser extrahiert nun die Binärmasken direkt über `out_binary_masks[idx]`. Durch diese Korrekturen konnte die Inferenz bei ca. 3,1 it/s vollständig stabilisiert werden.

und liefert mit dem Text-Prompt "cable" reproduzierbar **33 nicht-leere Masken** über den 100 Frames großen Testdatensatz hinweg.

Einfache Erklärung (für Laien/Nicht-Techniker verständlich):

Zuvor brach das System beim Versuch, das Kabel im Video automatisch zu verfolgen, ständig ab oder fand schlichtweg gar nichts. Das lag an drei anschaulichen Problemen:

- *Die Speicher-Überlastung (VRAM-Absturz):* Das Programm wollte alle KI-Bildanalysen für die Text-Prompts gleichzeitig im Grafikspeicher lösen. Das ist so, als ob man versucht, 10 dicke Bücher gleichzeitig auf einmal aufzuschlagen und querzulesen – das Gehirn (die GPU) ist überfordert. Indem wir dem System befehlen, die Sätze brav nacheinander zu verarbeiten (**Batch-Größe = 1**) und das zeitaufwendige „Vorab-Auswendiglernen“ ausschalten (**Kompilierung aus**), liest die GPU das Video nun stressfrei Bild für Bild. Der Speicherbedarf sank von über 12 GB auf gemütliche 4,5 GB.
- *Die Denkblockade (Der AssertionError):* Um Speicher zu sparen, hatten wir das Video anfangs in kurze 8-Bild-Häppchen zerlegt. Die KI kam bei der Übergabe der Häppchen jedoch völlig durcheinander, verlor den Faden und behauptete stur, die Teile würden logisch nicht mehr zusammenpassen. Wir lassen das Video nun einfach elegant am Stück durchlaufen und überlassen der KI die vollautomatische Verwaltung ihres Langzeitgedächtnisses.
- *Das Übersetzungs-Missverständnis (Leere Maskenergebnisse):* Die KI hat das Kabel im Video zwar erfolgreich gefunden und markiert, dies aber in einer neuen „fremden Sprache“ (einem veränderten Ausgabeformat) dokumentiert. Unser Empfänger-Skript suchte die Ergebnisse beharrlich am alten Ablageort und meldete fälschlicherweise „nichts gefunden“. Der neu geschriebene Parser übersetzt diese neue Struktur nun fehlerfrei, wodurch die erzeugten Kabelmasken exakt erfassungskonform abgespeichert werden können.

4. **[GELÖST: SAM 3.1 lauffähig nach Laufzeit-Modernisierung, Stand 06/2026]**

Die zuvor beschriebenen Lade- und Stabilitätsprobleme ließen sich auf eine **nicht explizit fixierte, zu niedrige Python-Laufzeit** (vormals Python 3.10) zurückführen.

Da SAM 3.1 moderne Laufzeitkomponenten voraussetzt, kam es ohne explizite Versionsfestlegung wiederholt zu Import- und Kompatibilitätsfehlern, sodass die Umgebung erst nach mehreren Korrekturen wieder vollständig lief. Die Lösung war eine konsequente **Pinnung der gesamten Laufzeit** in Container A: `nvidia/cuda:12.6.3-cudnn` mit **Python 3.12**, **PyTorch 2.7.0+cu126** und **torchvision 0.22.0**.

Drei konkrete Stolpersteine wurden dabei behoben: (i) Unter Ubuntu 24.04 verhindert die *externally-managed-environment*-Richtlinie (PEP 668) eine direkte `pip`-Installation; dies erfordert das Flag `--break-system-packages`. (ii) Beim Import des SAM-3.1-Pakets fehlten Laufzeitabhängigkeiten (`psutil`, `timm`), die explizit nachinstalliert werden mussten. (iii) Der Image-Pfad lädt standardmäßig das SAM-3-Gewicht; für SAM 3.1 muss der Checkpoint über `download_ckpt_from_hf(version="sam3.1")` ausdrücklich angefordert werden.

Ergänzend wurde ein **Einzelbild-Pfad** implementiert: Das Vorverarbeitungsskript erkennt nun automatisch, ob in `01_raw` ein Standbild (`.jpg/.jpeg/.png`) oder ein Video (`.mp4/.mov`) vorliegt, und nutzt für Bilder den text-geprompteten `build_sam3_image_model`.

with `Sam3Processor`. Dieser Pfad konnte auf einer statischen Testaufnahme mit dem Prompt “cable” reproduzierbar eine nichtleere Maske erzeugen. Der beim Laden weiterhin protokollierte `missing_keys`-Hinweis (vier `convs.3`-Einträge des Vision-Backbones) blieb dabei folgenlos und steht der Maskengenerierung nicht entgegen.

5. [\[GELÖST: Punktwolken-Kollaps durch COLMAP Multi-Model-Splits behoben, Stand 06/2026\]](#)

Bei unruhigen Kameraführungen oder homogenen Texturen bricht die geometrische Feature-Matching-Stufe von COLMAP die Rekonstruktion häufig in disjunkte Teilmodelle auf, welche sequenziell unter `sparse/0`, `sparse/1` usw. abgelegt werden. Das Dataloading-Skript von Segment-then-Splat (`prep_sts_scene.py`) war starr darauf konfiguriert, stets aus `sparse/0` zu lesen. Bei unvollständigen Initialisierungen enthielt dieses Modell jedoch oft nur minimale Artefakte (z.B. 2 registrierte Kameras mit gerade mal 1 spärlichen Raumpunkt auf dem Kabel), was im nachfolgenden 3DGS-Optimierer zum totalen geometrischen Kollaps des Trassenbereichs führte.

Als Lösung wurde ein **dynamischer, dateigrößenbasierter Best-Model-Resolver (Teilmodell-Wechsler)** implementiert. Dieser scannt vor dem STS-Training sämtliche von COLMAP generierten Unterordner und ermittelt über die Dateigröße der binären Punktwolkendatenbank (`points3D.bin`) vollautomatisch die dichteste und vollständigste Hauptrekonstruktion (z.B. `sparse/2` mit 123 Bildern und 89 validen Stützpunkten auf dem Kabel). Das Skript kopiert diese ausgewählte Wolke dynamisch als Haupt-Initialisierungs-Skelett nach `/data/05_3dgs/sparse/0`. Hierdurch konnte die nutzbare Kabelpunktwolke von 1 auf 89 Punkte vervielfacht werden, was das anschließende Splat-Wachstum und ein fehlerfreies Training vollständig stabilisiert.

Einfache Erklärung (für Laien/Nicht-Techniker verständlich):

Bei der 3D-Rekonstruktion versucht das Programm COLMAP, hunderte Fotos wie ein Puzzle passend aneinanderzureihen. Wenn die Drohne wackelt oder der Boden sehr eintönig aussieht, verliert das Programm den Faden. Statt dem *einen* großen Puzzle baut es heimlich mehrere kleine, voneinander getrennte Puzzle-Häufchen (Teilmodelle). Unser nachfolgendes Trainingsprogramm greift standardmäßig blind in das erste Körbchen (Modell 0). Blöderweise lag in diesem Körbchen diesmal nur ein winziges Randfragment, auf dem sich gerade mal ein einziger Punkt unseres Kabels befand – das eigentliche Kabel lag vollständig in Körbchen Nummer 2! Da das System somit nur 1 Kabel-Punkt zum Lernen hatte, löste sich das Kabel im 3D-Modell in Luft auf. Durch unsere Neuerung prüft ein intelligenter Suchgurt nun vorab die Größe aller Körbchen, identifiziert das vollwertige Hauptpuzzle (Modell 2 mit 89 Kabelpunkten) und übergibt dieses an das Training. Dadurch ist das Kabel nun stabil und lückenlos sichtbar.

6. **Das Georeferenzierungs-Paradoxon:** Da die objektspezifische Selektion (STS) nur das Kabel-Mesh isoliert, verschwinden im finalen 3D-Modell auch die am Boden ausgelegten Passpunkte (GCPs). Eine Post-Training Transformation des Modells durch manuelles Anklicken ist somit nicht anwendbar. Die Lösung ist eine *Pre-Training Georeferenzierung*: Die Open-Source-Software **CloudCompare** bietet hierfür ein ideales GUI. Da CloudCompare intern mit 32-Bit Single-Precision-Gleitkommazahlen arbeitet, führt eine direkte Eingabe von großen globalen UTM-Koordinaten ($> 10^5$) zu dramatischem Präzisionsverlust (1–10 cm). Daher wird eine *relative Georeferenzierung*

durchgeführt: Ein ausgewählter GCP dient als lokaler Ursprung (Ankerpunkt) mit den Koordinaten (0, 0, 0), und alle anderen GCPs werden relativ zu diesem berechnet ($GCP_{\text{relativ}} = GCP_{\text{UTM}} - GCP_{\text{Anker}}$). Man lädt die unmaskierte, spärliche SfM-Punktvolke aus COLMAP in CloudCompare, ordnet die Passpunkte den relativen GCP-Koordinaten manuell zu (*Point Picking*) und berechnet eine 4x4-Transformationsmatrix (Kombination aus 3x3-Rotation, Translation und optionaler Skalierung). SuGaR berechnet danach parallel das rohe, lokale 3D-Mesh. Anstatt jedoch das rechenintensive Millionen-Polygon-Mesh in globale Koordinaten zu überführen, wird zunächst DGtal auf dem lokalen Mesh ausgeführt. Die resultierende lokale Centerline (bestehend aus wenigen hundert Punkten) wird in Container E über ein kurzes Python-Hilfsskript mittels der 4x4-Transformationsmatrix multipliziert, um die Rotation und Skalierung anzuwenden, und danach durch Addition des Anker-GCPs in die globalen UTM-Koordinaten überführt.

7. [NEUE ERKENNTNIS: Datenformate & GIS-Export]

ArcGIS Pro unterstützt .ply-Geometrien nicht nativ als Feature-Layer. Zudem verursacht das .obj-Format bei großen UTM-Koordinaten signifikante Präzisionsverluste (*Vertex Jittering*), da es standardmäßig als *Single Precision* verarbeitet wird. Die Pipeline splittet den Datenexport daher gezielt in zwei Endprodukte auf:

- **Das analytische Endprodukt (Centerline):** Da DGtal (als diskrete Geometrie-Bibliothek) im lokalen Raum um den Ursprung (0,0,0) am präzisesten und speichereffizientesten arbeitet, wird die Centerline zuerst auf dem lokalen .ply-Mesh berechnet. Ein Python-Hilfsskript in Container E wendet die 4x4-Transformationsmatrix an, addiert den globalen Ankerpunkt und gibt eine georeferenzierte CSV-Datei aus. Die Konvertierung dieser CSV-Datei in standardisierte GIS-Formate (wie GeoJSON) erfolgt danach Open-Source über den **GDAL/ogr2ogr Post-Processing Container**.
- **Das visuelle Endprodukt (BIM-Mesh):** Falls TenneT das Kabel zusätzlich als 3D-Objekt visualisieren möchte, wird das lokale .obj-Mesh vorab (in CloudCompare oder über ein Python-Skript) mit dem Rotationsteil der 4x4-Matrix rotiert und skaliert. Anschließend wird die geänderte Geometrie in ArcGIS Pro über das nativ integrierte Geoprocessing-Tool "Import 3D Files" eingelesen und der Einfügepunkt (Insertion Point / Global Shift) direkt auf die UTM-Koordinaten des Anker-GCPs gesetzt, was den Präzisionsverlust des .obj-Formats elegant umgeht.

6 Wissenschaftliche Evaluationsmetriken

Für die wissenschaftliche und ingenieurstechnische Validierung der Pipeline werden zwei Kategorien von Metriken eingesetzt: geometrische Metriken für die Lagegenauigkeit des Endprodukts und photometrische Metriken für die visuelle Rekonstruktionsqualität des 3DGS-Modells.

6.1 Geometrische Metriken (Lage- und Höhengenaugigkeit)

Um nachzuweisen, dass die extrahierte Centerline die geforderte Toleranz von ± 10 cm einhält, werden die Abweichungen zu den hochpräzisen GNSS-Referenzmessungen quantifiziert.

- **B-Spline-Referenzkurve:** Die GNSS-Referenzmessungen liegen als diskrete, dreidimensionale Koordinatenpunkte $P_i = (x_i, y_i, z_i)$ vor. Ein direkter Punkt-zu-Punkt-Vergleich mit der dichter diskretisierten DGtal-Centerline ist fehleranfällig, da die Punkte nicht exakt korrespondieren. Daher wird eine kontinuierliche dreidimensionale **Basis-Spline-Kurve (B-Spline)** $S(t)$ durch die GNSS-Punkte interpoliert bzw. angenähert. Der B-Spline garantiert eine mathematisch glatte (zweifach stetig differenzierbare, C^2 -konsistente) Referenzachse des realen Kabels.
- **RMSE (Root Mean Square Error):** Der RMSE berechnet den quadratischen Mittelwert der euklidischen Abstände zwischen den Punkten C_j der extrahierten DGtal-Centerline und ihren jeweils orthogonalen Projektionen auf die B-Spline-Referenzkurve $S(t)$:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{j=1}^N \min_t \|C_j - S(t)\|^2}$$

Der RMSE gibt Aufschluss über die durchschnittliche systematische und stochastische Abweichung über den gesamten Verlauf der Trasse.

- **Hausdorff-Distanz:** Während der RMSE die mittlere Abweichung beschreibt, identifiziert die **Hausdorff-Distanz** d_H den worst-case Fehler (maximale lokale Abweichung). Sie ist definiert als das Maximum der minimalen Abstände zwischen der extrahierten Kurve C und der Referenzkurve S :

$$d_H(C, S) = \max \left\{ \sup_{c \in C} \inf_{s \in S} \|c - s\|, \sup_{s \in S} \inf_{c \in C} \|c - s\| \right\}$$

Für den Praxisbezug (TenneT) ist die Hausdorff-Distanz die entscheidende Metrik: Nur wenn $d_H(C, S) \leq 10$ cm gilt, ist garantiert, dass das Kabel an keiner einzigen Stelle der Trasse die ingenieurstechnische Toleranzgrenze überschreitet.

6.2 Photometrische Metriken (Rekonstruktionsqualität)

Diese Metriken messen ausschließlich die optische Qualität der 3D-Rekonstruktion. Sie werden **während des Trainings von Segment-then-Splat (STS)** auf gehaltenen Testbildern (Novel View Synthesis) berechnet, um sicherzustellen, dass die Geometrie der Szene visuell korrekt und ohne Artefakte erfasst wurde. Sie haben jedoch keinen direkten Bezug zur finalen UTM-Georeferenzierung.

- **PSNR (Peak Signal-to-Noise Ratio):** Misst das Verhältnis zwischen dem maximal möglichen Signalwert und dem störenden Rauschen im gerenderten Bild im Vergleich zum Ground-Truth-Foto. Der Wert wird logarithmisch in Dezibel (dB) angegeben. Höhere Werte (typischerweise > 28 dB im 3DGS) deuten auf eine präzise Rekonstruktion hin, wobei PSNR sehr empfindlich auf pixelgenaue Verschiebungen reagiert.

- **SSIM (Structural Similarity Index):** Bewertet im Gegensatz zu PSNR nicht nur den Pixelabstand, sondern die strukturelle Ähnlichkeit, Helligkeit und den Kontrast in lokalen Bildbereichen. SSIM liegt zwischen 0 und 1 (wobei 1 eine perfekte strukturelle Übereinstimmung beschreibt) und entspricht besser der menschlichen visuellen Wahrnehmung.
- **LPIPS (Learned Perceptual Image Patch Similarity):** Nutzt ein tiefes neuronales Netzwerk (z.B. VGG), um die perzeptuelle Ähnlichkeit auf Feature-Ebene zu vergleichen. Ein niedrigerer LPIPS-Wert zeigt an, dass das Rendering für das menschliche Auge (insbesondere bezüglich feiner Texturen und Schärfe) dem Originalfoto gleicht. Dies ist entscheidend, um zu prüfen, ob dünne Linienstrukturen verschwommen sind.

7 Zeit- und Scope-Planung

Projektarbeit (PA) – Fokus Machbarkeit (4 Wochen):

- Konfiguration der Docker-Architektur (Container A, B, C, D & E).
- Master-Skript Orchestrierung inkl. manuellem Breakpoint (CloudCompare).
- **Georeferenzierungs-Pipeline:** Implementierung der Transformationslogik (Berechnung der 4x4-Ähnlichkeitstransformationsmatrix in CloudCompare und automatisierte Transformation der extrahierten Centerline in Container E).
- Nachweis der Funktionalität der Segment-then-Splat Pipeline (Objekt-Loss) und Meshing (SuGaR) am Testdatensatz (Aluabspernung).
- *Optional:* Erste Voruntersuchungen zu SAM 2/3 und SAHI Tiling.

Bachelorarbeit (BA) – Fokus Skalierung & Metrik (10 Wochen):

- **Feldarbeit:** Drohnenflug sowie parallele GNSS-Referenzmessung der ausgelegten GCPs und der Kabel-Scheitelachse (*Centerline*) an der realen Trasse.
- Centerline-Extraktion via DGtal auf dem realen Kabel-Sub-Mesh.
- **Wissenschaftliche Evaluation (Metriken):** Berechnung des Root Mean Square Error (**RMSE**) und der maximalen **Hausdorff-Distanz** der extrahierten 3DGS-Centerline gegenüber einer interpolierten **Basis-Spline-Kurve (B-Spline)** aus den diskreten GNSS-Referenzmessungen, um die Einhaltung der ± 10 cm Toleranz nachzuweisen.
- Wirtschaftlichkeitsanalyse für TenneT.

Details der Parameterstudie und Sensitivitätsanalyse (BA)

Um die Robustheit und Präzision der entwickelten Pipeline unter realen Bedingungen nachzuweisen, wird im Rahmen der Bachelorarbeit eine systematische Parameterstudie durchgeführt. Dabei werden folgende Einflussfaktoren evaluiert:

- **GCP-Konfiguration (Anzahl und Verteilung):** Vergleich einer minimalen Konfiguration (3 GCPs) mit erweiterten Setups (5 bis 7 GCPs). Zudem wird die geometrische Anordnung untersucht: Lineare Platzierung (entlang des Kabelgrabens) versus stufenförmige/Zickzack-Verteilung (zur Stabilisierung der Rotationsachse um die Trassenachse).

- **Aufnahmeparameter (Flughöhe und Neigungswinkel):** Evaluierung des Einflusses der Flughöhe (5 m vs. 10 m vs. 15 m) auf die Ground Sampling Distance (GSD) und die Segmentierungsqualität. Vergleich von Nadir-Aufnahmen (90°) mit Oblique-Aufnahmen (15°–30° Neigung), um die zylindrische Geometrie der Kabelunterkante rauscharm zu rekonstruieren.
- **Segmentierungs- und Tracking-Modelle (Frontend-Vergleich):** Systematischer Vergleich zwischen dem SAM 3 Video Predictor, SAM 2 und SAHI Tiling (Slicing). Untersuchung des Einflusses der Frame-Extraktionsrate (z. B. jeder 5. vs. 10. vs. 20. Frame) auf die Rechengeschwindigkeit und die temporale Maskenkonsistenz zur Vermeidung von 3D-Rekonstruktionsfehlern.
- **Rekonstruktions- und Meshingparameter (STS & SuGaR):** Parametertuning der Poisson-Rekonstruktionstiefe (*Poisson Depth* 7, 8, 9 und 10) in SuGaR, um den optimalen Kompromiss zwischen der Erkennung des realen Kabeldurchmessers und der Rauschunterdrückung zu finden.
- **Kurvenglättung (DGtal-Postprocessing):** Vergleich verschiedener Glättungsalgorithmen (z. B. B-Spline-Approximation oder gleitender Mittelwert) zur Rauschfilterung der extrahierten 3D-Kurve.

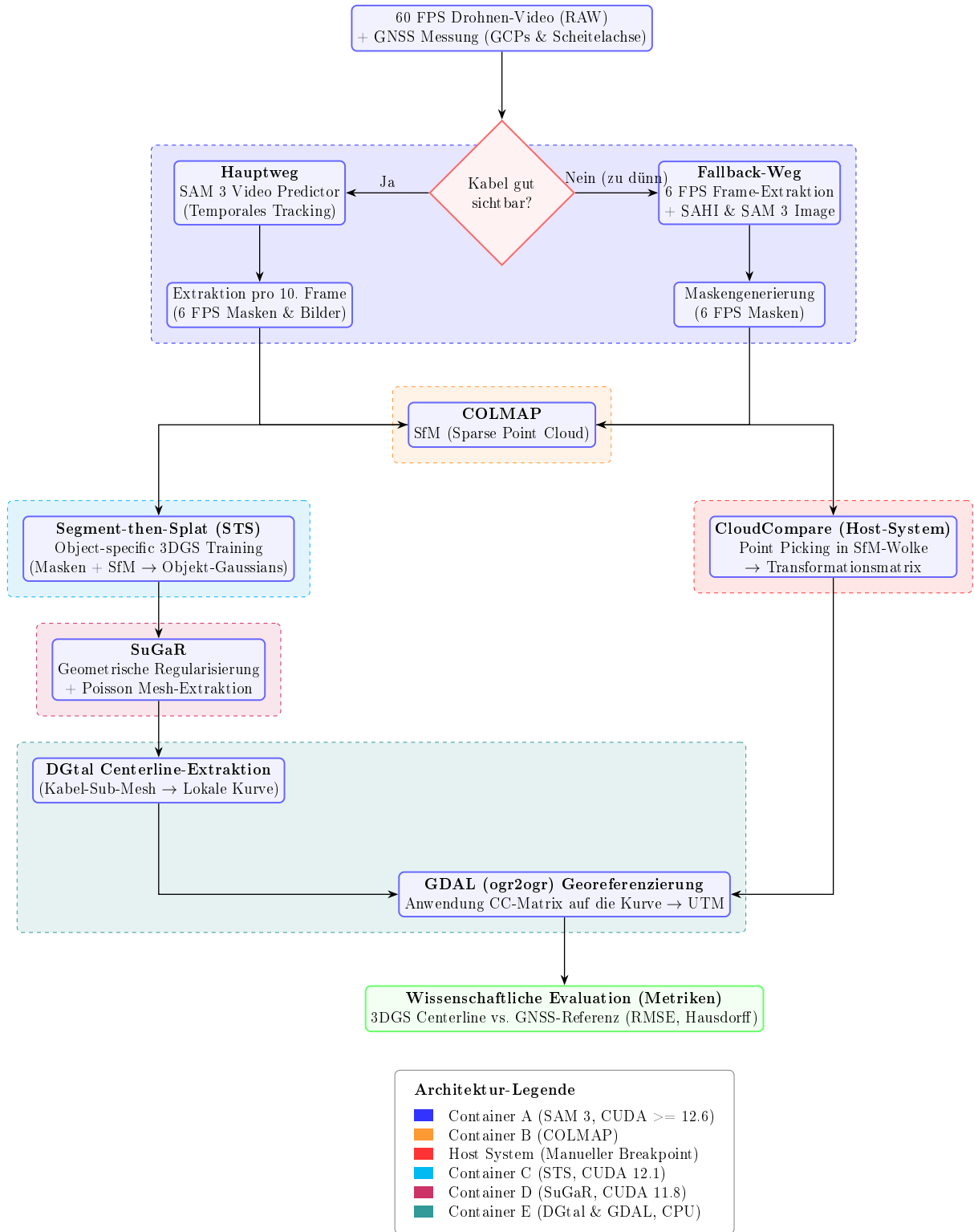


Abbildung 1: Microservice-Architektur (v4): 5-Container-Pipeline mit Segment-then-Splat für objektspezifisches Training und SuGaR für geometrisches Meshing.

8 Anhang: Single Source of Truth (SSOT) – Pipeline Installation

Dieses Dokument dient als zentrale Wahrheit (SSOT) für die Installation und Orchestrierung der gesamten 5-Container-Pipeline. Es fasst alle Hard- und Software-Abhängigkeiten zusammen, um Reproduzierbarkeit zu gewährleisten.

Hardware-Grundvoraussetzungen

- **GPU:** CUDA-kompatible Nvidia GPU (Compute Capability 7.0+).
- **VRAM:** 24 GB (erforderlich, um in Paper-Evaluations-Qualität zu trainieren).
- **OS:** Windows 10/11 mit WSL2 oder natives Ubuntu Linux 22.04.

Container A: SAM 3 (Segmentierung)

Zuständig für die 2D-Maskierung des Drohnenvideos. Dieser Container nutzt modernste Meta-Abhängigkeiten.

Requirements (laut offizieller Vorgabe):

- Python 3.10 or higher
- PyTorch 2.4 or higher (im Setup wird PyTorch mit der passenden CUDA-Laufzeitumgebung installiert)
- CUDA-kompatible GPU mit CUDA $\geq$ 12.6

Implementierungsentscheidung: Docker vs. Conda

Für die Ausführung der SAM 3.1 Codebasis wurde konzeptionell von einer reinen lokalen Conda-Umgebung Abstand genommen. Stattdessen läuft SAM 3.1 vollständig in einem dedizierten Docker-Container (`sam3-preprocess`).

Gründe gegen Conda und für Docker:

1. **CUDA-Konflikte:** SAM 3.1 verlangt moderne PyTorch-Versionen (z. B. 2.7.0 mit CUDA 12.6) und greift auf experimentelle C++ Extensions (`sam3_cu`) zurück. In einer lokalen Anaconda-Umgebung kommt es hier mit den bestehenden CUDA 11.8 Treibern von COLMAP/SuGaR ständig zu "Layer Mismatches" und Kompilierungsfehlern. Docker kapselt die CUDA 12.6 Abhängigkeiten sauber vom Host-System ab.
2. **Abhängigkeits-Knoten (Dependency Hell):** Das SAM 3.1 Release benötigt tiefgreifende Bibliotheken (`pycocotools`, `einops`, `timm`), die nativ im Container über `apt-get` und `pip` verlässlich zusammengestellt werden.
3. **Hardware-Starvation (Host-Safety):** Das Laden von SAM 3.1 in Full-HD (`sam3.1_multiplex`) absorbiert immense RAM- und CPU-Ressourcen beim Instanzieren des Video-Predictors. Im Container können via `docker-compose.yml` die CPU-Berechnungen strikt gedrosselt werden (`OMP_NUM_THREADS=8`), um Systemabstürze effektiv zu verhindern. Ein VRAM- und RAM-Offloading ist zusätzlich im inferierenden Python-Skript konfiguriert.
4. **Hugging Face Token-Sicherheit:** Über Docker-Secrets (`.env` / `HF_TOKEN`) wird die Authentifizierung für Gated Models sicher gehandhabt, ohne lokale Systemumgebungen zu belasten.

Installationsablauf (Docker):

Die Bereitstellung erfolgt vollautomatisch über das dedizierte **Dockerfile** auf Basis von **pytorch/pytorch:2.7.0-cuda12.6-cudnn9-devel**. Dies klonst das offizielle Repository, baut die C++ Extensions und cacht über das Hugging Face Hub das 3.5 GB schwere Multiplex-Gewicht persistent ins Host-Volume.

Ausführung & Masken-Export (Python API):

Hier ist das Python-Skript zur temporalen Propagation des Kabel-Prompts im Video. Da der SAM 3 Video Predictor im Tracking-Modus aus Konsistenzgründen nur eine einzelne, eindeutige Maske pro Frame propagiert (und nicht die drei hierarchischen Skalen des Image Predictors), generiert das Skript die für das Curriculum-Learning von Segment-then-Splat (STS) benötigten Stufen (**small, middle, default**) künstlich über morphologische Bildoperationen (Erosion und Dilation):

```
import os
import cv2
import numpy as np
from PIL import Image
from sam3.model_builder import build_sam3_video_predictor

# 1. Video Predictor auf GPU 0 initialisieren
predictor = build_sam3_video_predictor(gpus_to_use=[0])

# 2. Session mit dem Drohnen-Video starten
video_path = "/data/video.mp4"
response = predictor.handle_request(
    request=dict(type="start_session", resource_path=video_path)
)
session_id = response["session_id"]

# 3. Prompt setzen (z. B. Punkt-Prompt auf das Kabel in Frame 0)
predictor.handle_request(
    request=dict(
        type="add_prompt",
        session_id=session_id,
        frame_index=0,
        points=[[1024, 768]], # Beispielkoordinaten auf dem Kabel
        labels=[1]
    )
)

# 4. Stream-Propagation & Multi-Scale Export via Morphologie
output_dir = "/data/masks"
os.makedirs(output_dir, exist_ok=True)

# Definiere Kernel für Erosion und Dilation (Größe auf Kabeldurchmesser anpassen)
kernel = np.ones((5, 5), np.uint8)
```

```

# propagate_in_video liefert einen Generator
for response in predictor.handle_stream_request(
    request=dict(type="propagate_in_video", session_id=session_id)
):
    frame_idx = response["frame_index"]
    outputs = response["outputs"]
    obj_ids = outputs["out_obj_ids"]
    binary_masks = outputs["out_binary_masks"] # Shape: [num_objects, H, W]

    # Erzeuge einen Unterordner pro Frame-Index
    frame_dir = os.path.join(output_dir, f"frame_{frame_idx:05d}")
    os.makedirs(frame_dir, exist_ok=True)

    for idx, obj_id in enumerate(obj_ids):
        mask = binary_masks[idx] # Boolesches Array [H, W]
        if mask.any():
            # Konvertierung: True -> 255, False -> 0
            mask_uint8 = (mask * 255).astype(np.uint8)

            # 1. Middle (Original)
            img_middle = Image.fromarray(mask_uint8)
            img_middle.save(os.path.join(frame_dir, "middle.png"))

            # 2. Small (Erodiert für den stabilen Kabelkern)
            mask_small = cv2.erode(mask_uint8, kernel, iterations=1)
            img_small = Image.fromarray(mask_small)
            img_small.save(os.path.join(frame_dir, "small.png"))

            # 3. Default (Dilatiert für den Sicherheitsrand)
            mask_default = cv2.dilate(mask_uint8, kernel, iterations=1)
            img_default = Image.fromarray(mask_default)
            img_default.save(os.path.join(frame_dir, "default.png"))

# 5. Session sauber schließen
predictor.handle_request(
    request=dict(type="close_session", session_id=session_id)
)

```

Container B: COLMAP (Structure from Motion)

Generiert die Kameraposen und die Sparse Point Cloud aus den maskierten Drohnenbildern. Aufgrund der massiven Architekturänderungen (Einführung von GLOMAP) in Release 4.0.0, wird explizit zur stabileren Patch-Version **4.0.4** geraten, welche kritische Bugs (Speicherlecks, Race-Conditions in der Datenbank) behebt.

Quellen & Docker-Image:

- **Releases:** <https://github.com/colmap/colmap/releases>

- **Docker-Image:** `docker pull colmap/colmap:4.0.4-cuda`

Ausführung & PLY-Export (für CloudCompare):

Da COLMAP die Punktwolke standardmäßig in einem proprietären Binärformat (.bin) speichert, welches von CloudCompare nicht direkt gelesen werden kann, wird die Punktwolke direkt im Anschluss an die Rekonstruktion über den `model_converter` in eine standardisierte .ply-Datei exportiert. Um eine geometrische Verformung der Punktwolke (ebene Entartung/Kollaps) während der Rekonstruktion unkalibrierter Drohnenbilder zu verhindern, wird die Parametrisierung basierend auf modernsten Photogrammetrie-Workflows angepasst (SIMPLE_RADIAL Kameramodell, erzwungenes gemeinsames Objektiv via `single_camera 1` und sequentielles Matching).

```
# 1. Feature-Extraktion (Kameramodell einschränken & gleiches Objektiv forcieren)
colmap feature_extractor \
  --database_path /data/scene/database.db \
  --image_path /data/scene/images \
  --ImageReader.camera_model SIMPLE_RADIAL \
  --ImageReader.single_camera 1

# 2. Keypoint-Matching (Sequentiell mit Overlap statt Exhaustive für Videoüberhang)
colmap sequential_matcher \
  --database_path /data/scene/database.db \
  --SequentialMatching.overlap 15

# 3. Structure-from-Motion (Sparse Reconstruction)
mkdir -p /data/scene/sparse
colmap mapper \
  --database_path /data/scene/database.db \
  --image_path /data/scene/images \
  --output_path /data/scene/sparse

# 4. Export der Punktwolke als PLY-Format
colmap model_converter \
  --input_path /data/scene/sparse/0 \
  --output_path /data/scene/sparse/points3D.ply \
  --output_type PLY
```

Hintergrund zur mathematischen Optimierung (für „Dummis“ erklärt):

Wenn wir ein Drohnenvideo Frame-für-Frame in COLMAP einspeisen, stehen wir vor zwei massiven mathematischen Hürden:

1. **Die Overfitting-Falle (Zu viele Freiheitsgrade):** Wenn wir COLMAP ein extrem komplexes Kameramodell (wie OPENCV) geben, versucht der Algorithmus, für jedes der hunderte Bilder Brennweiten und Objektivkrümmungen völlig unabhängig voneinander zu schätzen. Das führt zu einer geometrischen Katastrophe: COLMAP „schummelt“ mathematisch, verbiegt die Parameter extrem und presst die gesamte 3D-Welt flach wie eine Flunder auf eine einzige Ebene zusammen, um Restfehler zu minimieren. Mit SIMPLE_RADIAL verbieten wir diese Schummelei, indem wir nur noch vier physikalische Kern-Parameter zulassen. Gleichzeitig zentrieren wir über `-ImageReader.single_camera 1` alle Rechenparameter auf ein gemeinsames Zentrum.

COLMAP kann die Tiefe dadurch nicht verlässlich berechnen und flacht die Punktwolke ab. Indem wir über `SAM3_FRAAMES_TEMPLATES` nur jedes z. B. 5. oder 10. Bildrekonstruktionsstandort. Dadurch schneiden sich die Sehstrahlen in einem gesunden, steilen Dreieck und zaubern die Struktur hervor.

2. Die Treffsicherheits-Falle (Falsche Übereinstimmungen): Das standardmäßige `exhaustive_matcher` vergleicht jedes Bild mit absolut jedem anderen Bild. Bei sich wiederholenden Texturen (z. B. Gleisen, Rohren oder Asphaltmustern) führt das zu fatalen Fehlverknüpfungen (False Loop Closures). Indem wir auf den `sequential_matcher` mit einem Overlap von 15 Frames umschalten, vergleicht COLMAP Bilder nur noch mit ihren unmittelbaren zeitlichen Nachbarn (im Video naheliegend). Das ist nicht nur hunderte Male schneller, sondern schützt die 3D-Punktwolke hochgradig effektiv vor geometrischer Verwerfung.
3. Die CLI-Versions-Falle (Veraltete Befehle): Beim Upgrade auf modernere, offizielle Docker-Images (wie `colmap/colmap:latest` ab Version 4.0) scheitern klassische, ältere Skripte an deprecated Parametern. Parameter wie `-SiftExtraction.use_gpu` oder `Breakpoint`: Nach diesem Schritt pausiert das Master-Skript automatisch. Die erzeugte Datei `points3D.ply` wird auf dem Host-System in CloudCompare geladen, um die manuelle Georeferenzierung via GCP-Point-Picking durchzuführen. Nach dem Speichern der Transformationsmatrix wird das Skript fortgesetzt.

Container C: Segment-then-Splat (Object-specific 3DGS)

Führt das objektspezifische 3DGS-Training durch. STS weist jedem initialen COLMAP-Gaussian eine Object-ID zu (basierend auf den SAM 3 Masken) und trainiert mit dem nativen Object-specific Loss ($\mathcal{L}_{obj}$). Das intern vorgesehene AutoSeg-SAM2 wird vollständig deaktiviert und durch die externen SAM 3 Masken aus Container A ersetzt. Hierfür wird der Dataloader (z. B. in `scene/dataset_readers.py` oder `scene/cameras.py`) so angepasst, dass er pro Frame die drei Masken-Dateien (`small.png`, `middle.png`, `default.png`) einliest und in das Dictionary `viewpoint_cam.object_masks` einspeist.

Requirements (Docker-Umgebung):

- **Base Image:** `nvidia/cuda:12.1.0-devel-ubuntu22.04`
- Python 3.10, PyTorch 2.3.1 mit CUDA 12.1
- 24 GB VRAM

Installationsablauf:

```
git clone https://github.com/luyr/Segment-then-Splat --recursive
cd Segment-then-Splat
```

```
conda create -n segment_then_splat python=3.10
conda activate segment_then_splat
pip install torch==2.3.1 torchvision==0.18.1 torchaudio==2.3.1 \
    --index-url https://download.pytorch.org/whl/cu121
pip install -r requirements.txt --no-build-isolation
```

Anpassung des Dataloaders und Trainingsablauf: Im Datensatz-Reader wird die folgende Logik implementiert, um die Maskentensoren direkt zu laden:


```

# Pseudo-Code zur Anpassung des Kamera-Ladevorgangs:
import torch
from PIL import Image

def load_object_masks(frame_dir):
    scales = ["small", "middle", "default"]
    object_masks = {}
    for scale in scales:
        mask_path = os.path.join(frame_dir, f"{scale}.png")
        if os.path.exists(mask_path):
            # Graustufen laden, Normalisieren und als Boolean-Tensor konvertieren
            mask_img = Image.open(mask_path).convert("L")
            mask_np = np.array(mask_img) > 127
            mask_tensor = torch.from_numpy(mask_np).bool() # Shape: [H, W]
            object_masks[scale] = [mask_tensor]
        else:
            object_masks[scale] = []
    return object_masks

# Zuweisung zur viewpoint_cam während der Szenen-Initialisierung:
# viewpoint_cam.object_masks = load_object_masks(f"/data/masks/frame_{frame_idx:05d}")

```

Ausführung des Trainings:

```

# 1. Object-specific Initialization (Zuweisung der initialen SfM-Punkte)
# STS prüft anhand der default/middle/small Masken die Punkt-Sichtbarkeiten
python ./helpers/object_specific_initialization.py \
    --scene_root /data/scene/

# 2. Training mit dem geänderten Dataloader und Object-specific Loss
python train.py -s /data/scene/ -m /data/output \
    --eval --iterations 40000 \
    --num_sample_objects 3 \
    --densify_until_iter 20000 \
    --partial_mask_iou 0.3

```

Wichtiger Hinweis: Durch die Entkopplung der Maskenberechnung läuft die CUDA-spezifische Optimization in Container C völlig autark von den CUDA-Anforderungen des SAM 3 Frontends in Container A.

Container D: SuGaR (Geometrische Regularisierung & Meshing)

Lädt den vortrainierten STS-Checkpoint und wendet die geometrische Regularisierung (DN-Consistency, Normal Alignment) sowie Poisson Surface Reconstruction an. Dies ist der sensitivste Container bezüglich CUDA-Versionen.

Exkurs: Conda innerhalb von Docker

Obwohl Docker bereits das Betriebssystem isoliert und Conda somit theoretisch redundant wäre, wird Miniconda in diesem Container explizit genutzt. Der Grund: 3DGS-Repositories

(wie SuGaR) setzen stark auf vorkompilierte Conda-Binaries (z.B. `pytorch-cuda`) und exakte Paket-Versionen. Durch die strikte Nutzung von Conda im Docker-Container werden fatale C++ Kompilierungsfehler beim Rasterizer-Modul präventiv vermieden.

Requirements (Docker-Umgebung):

- **Base Image:** `nvidia/cuda:11.8.0-devel-ubuntu22.04` (Dieses Image bringt das kompatible CUDA 11.8 SDK direkt mit).
- **C++ Compiler:** Installation via `apt-get install build-essential` (installiert GCC/G++, was unter Linux Visual Studio ersetzt und garantiert mit CUDA 11.8 kompatibel ist).
- 24 GB VRAM (erforderlich, um in Paper-Evaluations-Qualität zu trainieren).

Installationsablauf (Dockerfile-Struktur):

```
# 1. Base Image mit CUDA 11.8 & GCC
FROM nvidia/cuda:11.8.0-devel-ubuntu22.04

# 2. System-Abhängigkeiten installieren
ENV DEBIAN_FRONTEND=noninteractive
RUN apt-get update && apt-get install -y \
    build-essential \
    git \
    wget \
    curl \
    libgl1-mesa-glx \
    libglib2.0-0 \
    && rm -rf /var/lib/apt/lists/*

# 3. Miniconda installieren
RUN wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -O miniconda.sh
    bash miniconda.sh -b -p /opt/conda && \
    rm miniconda.sh
ENV PATH=/opt/conda/bin:$PATH

# 4. Conda Environment anlegen und Shell konfigurieren
RUN conda create --name sugar -y python=3.9
SHELL ["conda", "run", "-n", "sugar", "/bin/bash", "-c"]

# 5. PyTorch & CUDA 11.8 Bibliotheken installieren
RUN conda install pytorch==2.0.1 torchvision==0.15.2 torchaudio==2.0.2 \
    pytorch-cuda=11.8 -c pytorch -c nvidia -y && \
    conda install -c fvcore -c iopath -c conda-forge fvcore iopath -y && \
    conda install pytorch3d==0.7.4 -c pytorch3d -y && \
    conda install -c plotly plotly -y && \
    conda install -c conda-forge rich plyfile==0.8.1 jupyterlab nodejs ipywidgets -y

# 6. Repositories klonen und Submodule kompilieren
WORKDIR /workspace
```

```

RUN git clone https://github.com/Anttwo/SuGaR.git && \
  cd SuGaR && \
  pip install open3d PyMCubes && \
  pip install -e . && \
  cd gaussian_splatting/submodules/diff-gaussian-rasterization/ && \
  pip install -e . && \
  cd ../simple-knn/ && \
  pip install -e .

```

Conda-Umgebung im PATH systemweit als Standard setzen

```
ENV PATH=/opt/conda/envs/sugar/bin:$PATH
```

Details zur Build-Dauer und Kompilierungskomplexität (Container D):

In der praktischen Umsetzung stellte der Docker-Build für SuGaR eine signifikante Hürde dar, die mehrere Optimierungszyklen erforderte:

1. **Explizite CUDA-Architektur-Spezifikation (TORCH_CUDA_ARCH_LIST):** Um die Ausführbarkeit auf einer breiten Hardwarelandschaft (z.B. RTX 30er, 40er, Server-GPUs) abzusichern, mussten mehrere CUDA-Architekturen ("7.0;7.5;8.0;8.6;8.9;9.0") explizit beim Build hinterlegt werden. Der NVCC-Compiler kompiliert die C++ Extensions von `diff-gaussian-rasterization` und `simple-knn` für jede Architektur einzeln, was die Kompilierungszeit erheblich verlängert.
2. **Conda-Umgebung und Build-Isolation (--no-build-isolation):** Standardmäßig isoliert `pip` Builds in temporären virtuellen Umgebungen. Weil darin jedoch kein Zugriff auf die installierten PyTorch- und CUDA-Laufzeitbibliotheken von Conda besteht, kommt es zu fatalen Kompilierungsabbrüchen (z.B. bzgl. fehlender `torch`-Vektoren). Erst durch Hinzufügen von `--no-build-isolation` greift der Build-Prozess direkt auf den vorkonfigurierten Conda-Compiler-Stack zu.
3. **Parallelisierung via ninja-build:** Ohne eine parallele Kompilierungssteuerung fallen manche Python-Installer auf serielle Single-Thread-Kompilierung zurück, was zu extremen Timeouts oder Speicherüberläufen führt. Das Vorinstallieren des systemweiten Compilers `ninja-build` im Docker-Base-Image ermöglichte hierbei die maximal parallele Lastverteilung auf alle CPU-Kerne.

Automatisierte Punktwolkenfilterung zur Rauschminimierung und Qualitätssicherung (`filter_cable_pc.py`):

Ein schwerwiegender konzeptioneller Fehler bei der direkten Rekonstruktion mittels SuGaR ist die unkontrollierte Generierung von Hintergrund-Splats sowie von nahezu unsichtbaren „Dampf-Splats“ und schwarzen Rauschartefakten (Floating Artifacts bzw. Rauch-Gase) im leeren Raum. SuGaR besitzt keine eingebaute Logik, Hintergrundbereiche oder qualitativ minderwertige Splats zu verwerfen – ohne Filter arbeitet der Algorithmus an einer dichten, fehlerbehafteten Punktwolke. Ein eigens implementiertes Python-Hilfsprogramm `filter_cable_pc.py` wurde als Bindeglied (Scheduler Schritt 3.5) zwischen Phase 3 und Phase 4 geschaltet, das eine dreidimensionale Qualitätsfilterung durchführt:

- **1. Objektspezifische Selektion (Vordergrund-Isolierung):** Das Skript parst den binären PLY-Header des STS-Szenen-Outputs und liest die zugewiesenen Segmentierungs-Vektoren aus. Zu jedem 3D-Gaussian existieren die durch STS gelernten Objekt-IDs

(obj_id_s, obj_id_m, obj_id_l). Über die Parameter `--level m` und `--object_id 1` extrahiert das Skript gezielt die Kabel-Vordergrund-Splats (Standard voreingestellt als Klasse 1) und löscht unkorrelierte Hintergrundpunkte.

- **2. Opazitäts-Filterung (Eliminierung transparenter Floater):** Da während der mathematischen Optimierung viele fragile „Mogel-Splats“ mit extrem geringer Dichte entstehen, wird die Logit-Opazität α_{logit} des Punktes über die logistische Sigmoid-Funktion in eine echte physische Transparenzwirkung überführt:

$$\alpha_{\text{sigmoid}} = \frac{1}{1 + \exp(-\alpha_{\text{logit}})}$$

Punkte, deren berechnete $\alpha_{\text{sigmoid}} < 0.01$ beträgt, werden als unsichtbares Rauschen klassifiziert und vollständig verworfen.

- **3. Farbschwellenwert-Filterung (Entfernung schwarzer Dunstwolken):** 3DGS speichert Farbinformationen blickrichtungsabhängig in Form von sphärischen Harmonischen (SH). Die Koeffizienten der 0. Ordnung (f_dc_0, f_dc_1, f_dc_2) repräsentieren den diffusen Farbanteil. Um schwarze Splats (die das visuelle Bild stören und durch Rekonstruktionsunschärfen entstehen) mathematisch zu identifizieren, werden sie in den linearen RGB-Farbraum transformiert:

$$RGB = 0.5 + 0.28209479177387814 \cdot [f_dc_0, f_dc_1, f_dc_2]^T$$

Punkte, bei welchen sämtliche Farbkanäle unter einem einstellbaren Schwellenwert (Standard ≤ 0.08) liegen, werden zuverlässig als schwarzer Ruß oder Dunst identifiziert und aus der Geometrie eliminiert.

- **4. Dynamisches Quality-Gate (Sicherheits-Fallback):** Da ein totaler Datenverlust die nachfolgenden Meshing-Schritte sabotieren würde (SuGaR benötigt für die Oberflächenrekonstruktion eine Mindestanzahl an Stützpunkten), ist eine automatische Schutzlogik eingebaut. Sollte die Kombination aller Filterstufen fälschlicherweise 0 Punkte zurückliefern, verwirft das Skript die strengen Opazitäts- und Farbausschlüsse automatisch und reicht die rohe Objektsegmentierung als stabiles Fallback weiter.

Durch diesen erweiterten, mehrstufigen Qualitätsfilter-Lauf wird wirksam verhindert, dass SuGaR unvollständige oder durch schwarze Partikelblasen gestörte Oberflächen extrahiert, was die geometrische Präzision des finalen CAD-Modells drastisch verbessert.

Wichtige Best-Practices & Parameter-Tuning (SuGaR):

Für qualitativ hochwertige Rekonstruktionen (insb. dünne Kabelstrukturen) sind folgende Parameter entscheidend (siehe SuGaR Tips):

- **Regularisierung:** Nutze zwingend `-r "dn_consistency"` für die beste Mesh-Qualität.
- **Löcher im Mesh:** Wenn die Mesh-Bereinigung zu aggressiv ist, setze `vertices_density_quantile = 0.` in `sugar_extractors/coarse_mesh.py`.
- **Ellipsoide Artefakte (Bumps):** Reduziere die Poisson-Tiefe `poisson_depth` (z.B. von 10 auf 7 oder 8 in `coarse_mesh.py`), falls einzelne Gaussians sichtbar werden.
- **Große Szenen:** Bei weiten Distanzen sollten Custom Bounding Boxes (`-bboxmin` und `-bboxmax`) genutzt werden, um die Extraktion zu fokussieren.

Container E: DGtal & GDAL (Post-Processing)

Führt die Extraktion der Centerline auf dem isolierten Kabel-Sub-Mesh durch und überführt diese in georeferenzierte GIS-Formate (GeoJSON). Da DGtal eine komplexe C++ Bibliothek mit zahlreichen Abhängigkeiten (Eigen, CGAL, ITK) ist, wird direkt auf dem offiziellen `dgta:latest` Docker-Image aufgebaut.

Requirements & Repositories:

- **DGtal Release (v2.1):** <https://github.com/DGtal-team/DGtal/releases/tag/v2.1>
- **GDAL:** <https://github.com/OSGeo/gdal>
- **OGR2OGR Wrapper:** <https://github.com/wavded/ogr2ogr>

Docker Build & Ausführung:

```
# Offizielles DGtal Image bauen (im DGtal Repo Ordner)
docker build -t dgta:latest .
```

```
# Nachinstallation von GDAL (ogr2ogr) und Python-Paketen über ein abgeleitetes Docker-Image
FROM dgta:latest
USER root
RUN apt-get update && apt-get install -y gdal-bin libgdal-dev python3-pip python3-numpy
rm -rf /var/lib/apt/lists/*
USER digital
```

```
# Interaktive Ausführung mit Bind-Mount
docker run -it -v /host/path/data:/data --user=digital dgta-gdal:latest bash
cd /home/digital/git/DGtal/build/examples
# 1. DGtal Centerline-Extraktion auf dem lokalen Kabel-Mesh ausführen
# (Erzeugt /data/centerline_local.csv)
```

```
# 2. Ausführen des Python-Skripts zur Georeferenzierung
# (Wendet 4x4-Transformationsmatrix an und addiert UTM-Koordinaten des Anker-GCPs automatisch)
python /data/scripts/transform_centerline.py \
    --input_csv /data/centerline_local.csv \
    --matrix /data/matrix.txt \
    --anchor_txt /data/01_raw/anchor.txt \
    --output_csv /data/centerline_utm.csv
```

```
# 3. Konvertierung des UTM-Ergebnisses in standardisiertes GeoJSON (EPSG:25832)
ogr2ogr -f "GeoJSON" /data/centerline.geojson /data/centerline_utm.csv -a_srs EPSG:25832
```